ShelfEye: System Implementation

Nov 3, 2026 · @Vectis

Overview

ShelfEye replaces manual shelf counting with a battery-powered camera that counts stock and records every item taken. Each shelf gets one small camera unit built on a Raspberry Pi Zero. The unit photographs the shelf, uses the OpenAI vision API to identify and count the products, and lowers the count when an item is removed.

The Pi does all the processing and keeps its photos and records on its own SD card. It sends only the results (product, count, change, time) to a central server. A web dashboard reads from that server and shows staff live stock levels and recent activity.

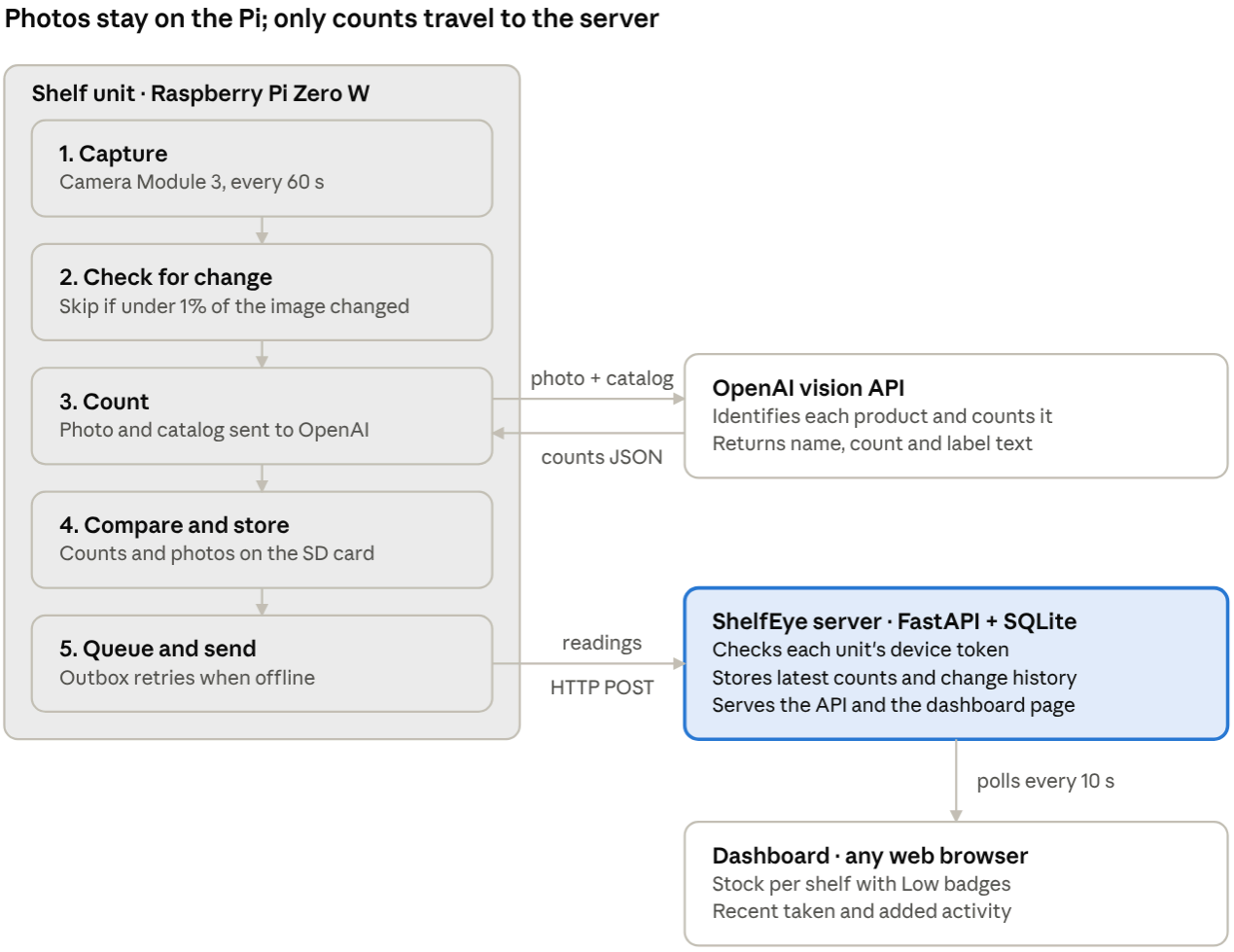
Hardware

Each shelf unit uses six components plus an OpenAI API key.

Component	Role in ShelfEye
Raspberry Pi Zero W (or Zero 2 W)	Runs the ShelfEye software, does the processing and connects to Wi-Fi. The plain Pi Zero has no Wi-Fi, so it can't be used.
Raspberry Pi Camera Module 3 (12MP)	Photographs the shelf. Autofocus keeps the labels sharp. Connects through the Pi Zero camera cable (22-pin to 15-pin).
3.7V 1200mAh Li-ion battery	Powers the unit away from a socket. It lasts a few hours under constant use, so mains or USB power is better for a permanent install.
Li-ion charge and boost module	Charges the battery and raises 3.7V to the 5V the Pi needs.
Rocker switch	Turns the unit on and off.
SD card	Holds the operating system, the ShelfEye software, the photos and the local database.
OpenAI API key	Lets the Pi send photos to the OpenAI vision API for counting. It's stored on the Pi in a file only ShelfEye can read.

System architecture

ShelfEye has three parts: the shelf unit, the server and the dashboard. The OpenAI API is called only by the shelf unit.



ShelfEye architecture · shelf unit, OpenAI, server, dashboard

Each shelf unit runs the full pipeline locally and sends only small text readings onward. More shelves just means more units posting to the same server.

Device software

The Pi runs one Python program in a loop, and it starts automatically on boot. It calls the paid API only when the shelf has actually changed. Each pass of the loop works like this:

1. **Capture.** The camera takes a photo every 60 seconds and saves it to the SD card.
2. **Check for change.** The Pi shrinks the photo to a 160×90 greyscale copy and compares it with the last photo it analysed. If less than 1% of the image changed, it skips the API call and deletes the photo.

3. **Wait for stillness.** It analyses only when two photos in a row match, so a hand reaching into the shelf isn't counted.
4. **Count with OpenAI.** It sends a reduced copy of the photo with the shelf's product list (the catalog). OpenAI returns structured JSON: each product's name, count and the label text it read.
5. **Compare and store.** It compares the new counts with the previous ones in a SQLite database on the SD card and records each change, e.g. "Paracetamol 500mg: 6 → 5 (−1)".
6. **Queue and send.** It saves the reading to an outbox on the SD card, then pushes the outbox to the server. If Wi-Fi is down, readings wait on the card and are sent once it's back.
7. **Re-check.** Every 30 minutes it re-counts even without a change, to correct any drift.

The catalog matters: it makes the model use the same product names every time, so counts can be compared from one reading to the next. Analysed photos stay on the SD card for 7 days, then they're deleted.

The main settings live in one config file on the Pi:

Setting	Default	What it controls
`CAPTURE_INTERVAL_SECONDS`	60	Time between photos
`CHANGE_THRESHOLD`	0.01	Share of the image that must change before calling OpenAI
`RECHECK_MINUTES`	30	Forced re-count interval
`KEEP_IMAGES_DAYS`	7	How long analysed photos are kept
`OPENAI_MODEL`	gpt-4.1-mini	Which OpenAI vision model is used

How the server comes in

The server is the meeting point between the shelf units and the people using the dashboard. Each Pi only knows about its own shelf. The server collects readings from every unit into one database, so staff see all shelves in one place without connecting to any Pi directly.

It does three jobs:

- **Receives readings** from each Pi over HTTP, after checking the device token.
- **Stores** the latest count per product per shelf, plus the full history of changes.

- **Serves** the data to the dashboard through a small read-only API, and serves the dashboard page itself.

The server is a Python FastAPI app with a SQLite database. It needs no image processing and no OpenAI access, so it can run on any small computer, a laptop or a low-cost cloud server.

What a Pi sends

After each analysis the Pi posts one reading. It carries the full shelf snapshot and the changes since the last reading:

```
{
  "reading_id": "7f3c…",
  "captured_at": "2026-11-03T10:15:00+00:00",
  "image": "images/2026-11-03/101500.jpg",
  "items": [{"name": "Paracetamol 500mg", "count": 5, "label_text":
"Paracetamol 500mg Tablets"}],
  "changes": [{"name": "Paracetamol 500mg", "previous": 6, "current": 5,
"delta": -1}]
}
```

The photo itself stays on the Pi. The `image` field only records which file on the SD card the reading came from.

API endpoints

Method and path	Used by	Purpose
`POST /api/nodes/{node_id}/readings`	Pi	Submit a reading. Requires the device token.
`GET /api/inventory`	Dashboard	Current count of every product on every shelf
`GET /api/events?limit=50`	Dashboard	Recent changes, newest first
`GET /api/nodes`	Dashboard	Each shelf unit and when it last reported
`GET /`	Browser	The dashboard page

Database tables

Table	Holds
`nodes`	Each shelf unit and the time it last reported
`readings`	One row per reading received, keyed by `reading_id`
`inventory`	Latest count and label text per product per shelf
`events`	Every change: product, previous count, new count, difference, time

Reliability and security

- **Device token:** each Pi sends a shared secret in the `Authorization` header. Readings without it are rejected with 401, so nobody else can post fake counts.
- **No duplicates:** if a Pi resends a reading because a reply was lost, the server spots the repeated `reading_id` and ignores it.
- **Full snapshots:** each reading replaces that shelf's inventory, so the server always matches the Pi's latest view even after missed messages.
- **Offline tolerance:** if the server is unreachable, readings queue on the Pi's SD card and arrive in order once it's back.

Dashboard

Staff open the dashboard in any web browser, on a phone or a computer. The server hosts it, so there's nothing to install. It refreshes every 10 seconds and shows:

- **Stock on shelves:** every product on every shelf with its current count and when it was last updated. Products with 2 or fewer units get a red "Low" badge.
- **Recent activity:** the latest changes, e.g. "1 taken · Paracetamol 500mg (6 → 5)", with the shelf and time.
- **Status line:** how many shelf units are reporting, and a warning if the server can't be reached.

The dashboard reads only from the server's GET endpoints and can't change any counts.

Why OpenAI instead of YOLO, and limitations

ShelfEye counts with the OpenAI vision API, not a YOLO model. The earlier design flowchart showed YOLO. We changed it for three reasons:

- **The Pi Zero is too weak for YOLO.** It has a single-core CPU (on the Zero W) and 512 MB of RAM, so even a small model would be very slow per image.
- **YOLO would need training on every product.** Out of the box it knows generic objects like "bottle", not "Paracetamol 500mg". It would need hundreds of labelled photos per product, and retraining for each new one.
- **OpenAI can read labels.** Adding a new product only means adding its name to the catalog.

Known limitations:

- **Only visible items are counted.** Boxes hidden behind the front row are missed, so the count is visible stock.
- **Accuracy drops with large piles** of identical items. It works best with small numbers of clearly visible units per product.
- **Placement matters.** The camera must be fixed and the shelf evenly lit. A moved camera makes every frame look like a change.
- **Each analysis is a paid API call.** Change detection keeps calls to when the shelf changes, plus one re-check every 30 minutes.
- **Internet is required** for counting. Without it, the Pi keeps taking photos but can't count until the connection returns.

YOLO could still be added later on the server, either to cut API costs once enough shelf photos exist to train it, or to detect people for the "who took it" feature.

Deployment and next steps

Setting up a shelf unit takes four steps:

1. Flash Raspberry Pi OS Lite onto the SD card with Raspberry Pi Imager, entering the Wi-Fi details and turning on SSH.
2. Copy the ShelfEye device folder to the card's boot partition, with the config file (API key, server address, device token) and the shelf's product catalog.
3. Boot the Pi, connect over SSH and run `install.sh`. It installs the packages, sets up the config and starts ShelfEye on every boot.
4. Start the server on a computer on the same network and open the dashboard in a browser.

The full software pipeline has been tested end to end with the OpenAI response simulated: change detection, counting, local storage, upload, server storage and the dashboard. Still to do:

- [ ] Run a real OpenAI count on photos of the actual shelf and check accuracy

- [ ] Test the camera on the Pi and settle on a camera position and lighting
- [ ] Measure battery life and decide between battery and mains power
- [ ] Choose where the server will run permanently
- [ ] Update the earlier architecture diagrams and flowchart to match this design